

Vue

Violet

2026-04-28

Chapter 0

目 录

I	Vue 3 核心基础
1.	Vue 3 入门与环境搭建 ····· 7
2.	Composition API 核心 ····· 8
3.	组件系统 ····· 9
3.1.	组件注册 ····· 9
3.1.1.	全局注册 ····· 9
3.1.2.	局部注册 ····· 9
3.1.3.	自动导入 (unplugin-vue-components) ····· 10
3.2.	Props 与 Emits ····· 11
3.2.1.	Props 类型声明 ····· 11
3.2.2.	Props 验证 ····· 12
3.2.3.	Emits 类型声明 ····· 13
3.2.4.	v-model 双向绑定 ····· 14
3.3.	插槽 (Slots) ····· 15
3.3.1.	默认插槽 ····· 15
3.3.2.	具名插槽 ····· 16
3.3.3.	作用域插槽 ····· 17
3.4.	异步组件 ····· 18
3.4.1.	defineAsyncComponent ····· 18
3.4.2.	Suspense ····· 19
3.5.	组件通信 ····· 20
3.5.1.	Props/Emits (父子通信) ····· 20
3.5.2.	Provide/Inject (跨层级通信) ····· 20
3.5.3.	Mitt 事件总线 (兄弟组件通信) ····· 21
3.5.4.	Pinia (全局状态管理) ····· 22
4.	指令与过渡动画 ····· 24
4.1.	内置指令 ····· 24
4.1.1.	v-if / v-show ····· 24
4.1.2.	v-for ····· 25
4.1.3.	v-bind / v-on ····· 25
4.1.4.	v-model ····· 26

4.1.5. v-slot	28
4.2. 自定义指令	28
4.2.1. 指令注册	28
4.2.2. 钩子函数	29
4.2.3. 实战示例	30
4.3. 过渡系统	32
4.3.1. <Transition> 基础	32
4.3.2. CSS 过渡/动画	32
4.3.3. JavaScript 钩子	34
4.4. 列表过渡	35
4.4.1.	35
4.4.2. FLIP 动画	36
4.4.3. 性能优化	38

II

Vue 官方生态

5. Vue Router 路由管理	40
6. Pinia 状态管理	41
7. 数据持久化	42
8. HTTP 请求与 API 管理	43

III

现代 UI 库与 Shadcn-vue

9. UI 库演进与 Shadcn 崛起	45
9.1. 传统 UI 库困境	45
9.1.1. 体积臃肿	45
9.1.2. 样式覆盖困难	45
9.1.3. 定制性差	46
9.2. Shadcn 理念	46
9.2.1. 不是组件库而是代码集合	46
9.2.2. Copy-Paste 工作流	47
9.2.3. 基于 Tailwind CSS	48
9.3. Shadcn 优势	48
9.3.1. 零运行时开销	48
9.3.2. 无障碍访问 (a11y)	49
9.3.3. 主题定制	50
9.3.4. Tree-Shaking 友好	50
9.4. 生态对比	51
9.4.1. Radix UI (React Headless UI)	51
9.4.2. Reka UI (Vue Headless UI)	51
9.4.3. Shadcn/ui (React) vs Shadcn-vue (Vue)	52

10.	Shadcn-vue 快速上手	54
11.	Shadcn-vue 核心组件	55
12.	Shadcn-vue 高级用法	56
13.	其他现代 UI 方案	57
IV	工程化与最佳实践	
14.	TypeScript 集成	59
15.	Vite 构建优化	60
16.	测试与质量保障	61
17.	性能优化	62
V	Vue 2 兼容与迁移	
18.	Options API 回顾	64
19.	Vue 2 → Vue 3 迁移	65



Vue 3 核心基础

1. Vue 3 入门与环境搭建 ·····	7
2. Composition API 核心 ·····	8
3. 组件系统 ·····	9
4. 指令与过渡动画 ·····	24

Chapter 1

Vue 3 入门与环境搭建

Chapter 2

Composition API 核心

Chapter 3

组件系统

组件是 Vue 应用的核心构建块。Vue 3 提供了强大的组件系统，支持灵活的注册方式、类型安全的 Props/Emits、以及多种通信机制。

1 组件注册

1.1 全局注册

```
1 // main.ts
2 import { createApp } from 'vue'
3 import App from './App.vue'
4
5 // 全局注册组件
6 import BaseButton from './components/BaseButton.vue'
7 import BaseInput from './components/BaseInput.vue'
8
9 const app = createApp(App)
10 app.component('BaseButton', BaseButton)
11 app.component('BaseInput', BaseInput)
12 app.mount('#app')
```

使用：

```
1 <!-- 任何组件中都可以直接使用，无需导入 -->
2 <template>
3   <BaseButton>点击</BaseButton>
4   <BaseInput v-model="value" />
5 </template>
```

优缺点：

优点	缺点
使用方便，无需导入	所有组件都会打包进 bundle
适合基础组件	可能导致命名冲突
减少重复导入	不利于 Tree-Shaking

CAUTION

全局注册适合少量基础组件（Button、Input 等），不建议大量使用。

1.2 局部注册

```
1 <script setup lang="ts">
2   import MyComponent from './MyComponent.vue'
3   import AnotherComponent from './AnotherComponent.vue'
4
5   // 自动注册 (<script setup>)
6   // 只需导入，无需额外注册
7 </script>
8
9 <template>
10   <MyComponent />
11   <AnotherComponent />
12 </template>
```

传统写法：

```
1 <script>
2   import MyComponent from './MyComponent.vue'
3
4   export default {
5     components: {
6       MyComponent // 手动注册
7     }
8   }
9 </script>
```

优势：

- 更好的 Tree-Shaking：未使用的组件不会被打包
- 明确的依赖关系：可以清楚看到组件使用了哪些子组件
- 避免命名冲突：组件名只在当前作用域有效

TIP

推荐使用局部注册，配合自动导入插件提升开发体验。

1.3 自动导入 (unplugin-vue-components)

```
1 # 安装
2 npm install -D unplugin-vue-components
```

```
1 // vite.config.ts
2   import Components from 'unplugin-vue-components/vite'
3
4   export default defineConfig({
5     plugins: [
6       vue(),
7       Components({
8         // 自动导入 src/components 目录下的组件
9         dirs: ['src/components'],
10        // 生成类型声明
11        dts: true,
12        // 支持 UI 库
13        resolvers: [
14          // Element Plus
```

```
15     ElementPlusResolver(),
16     // Shadcn-vue
17     // 不需要 resolver, 因为代码在项目中
18   ],
19   }),
20 ],
21 })
```

使用:

```
1 <script setup lang="ts">
2 // 无需导入! 自动识别
3 </script>
4
5 <template>
6   <BaseButton>自动导入的组件</BaseButton>
7   <MyCustomComponent /> <!-- 只要在 components 目录下 -->
8 </template>
```

生成的类型声明:

```
1 // components.d.ts (自动生成)
2 declare module 'vue' {
3   export interface GlobalComponents {
4     BaseButton: typeof import('./src/components/BaseButton.vue')['default']
5     MyCustomComponent: typeof import('./src/components/MyCustomComponent.vue')['default']
6   }
7 }
```

NOTE

自动导入插件结合了全局注册的便利性和局部注册的 Tree-Shaking 优势。

2 Props 与 Emits

2.1 Props 类型声明

```
1 <script setup lang="ts">
2 interface Props {
3   title: string
4   count?: number // 可选
5   disabled?: boolean
6   items: Array<{ id: number; name: string }>
7 }
8
9 // 带默认值
10 withDefaults(defineProps<Props>(), {
11   count: 0,
12   disabled: false,
13 })
14 </script>
```

```
15
16 <template>
17   <h1>{{ title }}</h1>
18   <p>Count: {{ count }}</p>
19 </template>
```

运行时声明（不使用 TypeScript）：

```
1  <script setup>
2    defineProps({
3      title: {
4        type: String,
5        required: true,
6      },
7      count: {
8        type: Number,
9        default: 0,
10     },
11     disabled: Boolean,
12     items: {
13       type: Array,
14       default: () => [],
15     },
16   })
17 </script>
```

TIP

推荐使用 TypeScript + `defineProps<T>()`，获得完整的类型检查和 IDE 支持。

2.2 Props 验证

```
1  <script setup lang="ts">
2    interface Props {
3      status: 'success' | 'warning' | 'error' // 字面量联合类型
4      size: 'sm' | 'md' | 'lg'
5      color?: string
6    }
7
8    const props = defineProps<Props>()
9
10   // 自定义验证逻辑
11   import { watchEffect } from 'vue'
12
13   watchEffect(() => {
14     if (props.status === 'error' && !props.color) {
15       console.warn('Error status should have a color')
16     }
17   })
18 </script>
```

运行时验证：

```
1 <script setup>
2   defineProps({
3     status: {
4       type: String,
5       validator: (value) => {
6         return ['success', 'warning', 'error'].includes(value)
7       },
8     },
9     age: {
10      type: Number,
11      validator: (value) => {
12        return value >= 0 && value <= 150
13      },
14    },
15  })
16 </script>
```

2.3 Emits 类型声明

```
1 <script setup lang="ts">
2   // 定义 emit 事件
3   const emit = defineEmits<{
4     (e: 'update', value: string): void
5     (e: 'change', id: number, name: string): void
6     (e: 'delete', id: number): void
7   }>()
8
9   // 触发事件
10  function handleClick() {
11    emit('update', 'new value')
12    emit('change', 1, 'Item 1')
13  }
14
15  function handleDelete(id: number) {
16    emit('delete', id)
17  }
18 </script>
19
20 <template>
21   <button @click="handleClick">更新</button>
22   <button @click="handleDelete(1)">删除</button>
23 </template>
```

父组件使用：

```
1 <script setup lang="ts">
2   import ChildComponent from './ChildComponent.vue'
3
4   function handleUpdate(value: string) {
5     console.log('Updated:', value)
6   }
7
```

```
8  function handleChange(id: number, name: string) {
9    console.log(`Changed: ${id} - ${name}`)
10 }
11 </script>
12
13 <template>
14   <ChildComponent
15     @update="handleUpdate"
16     @change="handleChange"
17     @delete="(id) => console.log('Deleted:', id)"
18   />
19 </template>
```

NOTE

TypeScript 声明的 emits 提供完整的类型检查，IDE 会自动补全事件名和参数。

2.4 v-model 双向绑定

```
1  <!-- 子组件 -->
2  <script setup lang="ts">
3    const props = defineProps<{
4      modelValue: string
5    }>()
6
7    const emit = defineEmits<{
8      (e: 'update:modelValue', value: string): void
9    }>()
10
11    function updateValue(event: Event) {
12      const target = event.target as HTMLInputElement
13      emit('update:modelValue', target.value)
14    }
15  </script>
16
17  <template>
18    <input :value="modelValue" @input="updateValue" />
19  </template>
```

父组件使用：

```
1  <script setup lang="ts">
2    import { ref } from 'vue'
3    import CustomInput from './CustomInput.vue'
4
5    const inputValue = ref('Hello')
6  </script>
7
8  <template>
9    <CustomInput v-model="inputValue" />
10    <p>Value: {{ inputValue }}</p>
11  </template>
```

多个 v-model:

```
1  <!-- 子组件 -->
2  <script setup lang="ts">
3    defineProps<{
4      modelValue: string
5      modelModifiers?: Record<string, boolean>
6      title: string
7    }>()
8
9    defineEmits<{
10      (e: 'update:modelValue', value: string): void
11      (e: 'update:title', value: string): void
12    }>()
13  </script>
14
15  <!-- 父组件 -->
16  <template>
17    <CustomInput
18      v-model="inputValue"
19      v-model:title="pageTitle"
20    />
21  </template>
```

TIP

Vue 3 支持多个 v-model，通过不同的 modifier 实现更灵活的双向绑定。

3 插槽 (Slots)

3.1 默认插槽

```
1  <!-- Card.vue -->
2  <template>
3    <div class="card">
4      <div class="card-header">
5        <slot name="header">默认标题</slot>
6      </div>
7
8      <div class="card-body">
9        <slot>默认内容</slot>
10     </div>
11
12     <div class="card-footer">
13       <slot name="footer">默认底部</slot>
14     </div>
15   </div>
16 </template>
```

使用:

```
1 <Card>
2   <template #header>
3     <h2>自定义标题</h2>
4   </template>
5
6   <p>自定义内容</p>
7
8   <template #footer>
9     <button>操作按钮</button>
10  </template>
11 </Card>
```

3.2 具名插槽

```
1 <!-- Layout.vue -->
2 <template>
3   <div class="layout">
4     <header>
5       <slot name="header"></slot>
6     </header>
7
8     <main>
9       <slot name="default"></slot>
10    </main>
11
12    <aside>
13      <slot name="sidebar"></slot>
14    </aside>
15
16    <footer>
17      <slot name="footer"></slot>
18    </footer>
19  </div>
20 </template>
```

使用：

```
1 <Layout>
2   <template #header>
3     <NavBar />
4   </template>
5
6   <template #default>
7     <RouterView />
8   </template>
9
10  <template #sidebar>
11    <SideMenu />
12  </template>
13
14  <template #footer>
15    <Footer />
```



```
16   </template>
17 </Layout>
```

NOTE

具名插槽让组件布局更加灵活，适合构建页面模板。

3.3 作用域插槽

```
1  <!-- DataTable.vue -->
2  <script setup lang="ts">
3    interface Row {
4      id: number
5      name: string
6      email: string
7    }
8
9    defineProps<{
10      data: Row[]
11    }>()
12  </script>
13
14  <template>
15    <table>
16      <thead>
17        <tr>
18          <th>ID</th>
19          <th>Name</th>
20          <th>Email</th>
21          <th>Actions</th>
22        </tr>
23      </thead>
24      <tbody>
25        <tr v-for="row in data" :key="row.id">
26          <td>{{ row.id }}</td>
27          <td>{{ row.name }}</td>
28          <td>{{ row.email }}</td>
29          <td>
30            <!-- 作用域插槽：将 row 数据传递给父组件 -->
31            <slot name="actions" :row="row"></slot>
32          </td>
33        </tr>
34      </tbody>
35    </table>
36  </template>
```

使用：

```
1  <script setup lang="ts">
2    import { ref } from 'vue'
3    import DataTable from './DataTable.vue'
4
5    const users = ref([
```

```
6   { id: 1, name: 'Alice', email: 'alice@example.com' },
7   { id: 2, name: 'Bob', email: 'bob@example.com' },
8 ]})
9
10 function handleEdit(row: any) {
11   console.log('Edit:', row)
12 }
13
14 function handleDelete(row: any) {
15   console.log('Delete:', row)
16 }
17 </script>
18
19 <template>
20   <DataTable :data="users">
21     <template #actions="{ row }">
22       <button @click="handleEdit(row)">编辑</button>
23       <button @click="handleDelete(row)">删除</button>
24     </template>
25   </DataTable>
26 </template>
```

TIP

作用域插槽让父组件可以访问子组件的数据，实现高度定制化的渲染。

4 异步组件

4.1 defineAsyncComponent

```
1 <script setup lang="ts">
2   import { defineAsyncComponent } from 'vue'
3
4   // 基本用法
5   const AsyncComponent = defineAsyncComponent(() =>
6     import('./HeavyComponent.vue')
7   )
8
9   // 带加载状态
10  const AsyncComponentWithLoading = defineAsyncComponent({
11    loader: () => import('./HeavyComponent.vue'),
12    loadingComponent: LoadingSpinner,
13    delay: 200, // 延迟显示 loading (避免闪烁)
14    timeout: 3000, // 超时时间
15    errorComponent: ErrorDisplay,
16    onError(error, retry, fail, attempts) {
17      if (attempts <= 3) {
18        retry() // 重试
19      } else {
20        fail() // 失败
```

vue

```
21   }
22 },
23 })
24 </script>
25
26 <template>
27   <AsyncComponent />
28   <AsyncComponentWithLoading />
29 </template>
```

NOTE

异步组件适合大型组件或很少使用的组件，可以减小初始 bundle 大小。

4.2 Suspense

```
1  <!-- Parent.vue -->
2  <script setup lang="ts">
3    import AsyncChild from './AsyncChild.vue'
4  </script>
5
6  <template>
7    <Suspense>
8      <!-- 默认内容（异步组件加载完成后显示） -->
9      <template #default>
10        <AsyncChild />
11      </template>
12
13      <!-- 后备内容（加载过程中显示） -->
14      <template #fallback>
15        <div class="loading">
16          <p>加载中...</p>
17        </div>
18      </template>
19    </Suspense>
20  </template>
```

AsyncChild.vue:

```
1  <script setup lang="ts">
2    // async setup 自动被 Suspense 捕获
3    const data = await fetch('/api/data').then(res => res.json())
4  </script>
5
6  <template>
7    <div>{{ data }}</div>
8  </template>
```

CAUTION

Suspense 目前是实验性功能，API 可能会在未来版本中调整。

5 组件通信

5.1 Props/Emits (父子通信)

```
1 <!-- 父组件 -->
2 <script setup lang="ts">
3   import { ref } from 'vue'
4   import Child from './Child.vue'
5
6   const message = ref('Hello from parent')
7
8   function handleMessageFromChild(msg: string) {
9     console.log('Received:', msg)
10  }
11 </script>
12
13 <template>
14   <Child
15     :message="message"
16     @child-message="handleMessageFromChild"
17   />
18 </template>
```

```
1 <!-- 子组件 -->
2 <script setup lang="ts">
3   defineProps<{
4     message: string
5   }>()
6
7   const emit = defineEmits<{
8     (e: 'child-message', msg: string): void
9   }>()
10
11   function sendMessage() {
12     emit('child-message', 'Hello from child')
13   }
14 </script>
15
16 <template>
17   <p>{{ message }}</p>
18   <button @click="sendMessage">发送消息</button>
19 </template>
```

5.2 Provide/Inject (跨层级通信)

```
1 <!-- 祖先组件 -->
2 <script setup lang="ts">
3   import { provide, ref } from 'vue'
4
5   const theme = ref('light')
```

```
6  const user = ref({ name: 'Alice' })
7
8  // 提供数据
9  provide('theme', theme)
10 provide('user', user)
11
12 // 提供方法
13 provide('setTheme', (newTheme: string) => {
14   theme.value = newTheme
15 })
16 </script>
```

```
1  <!-- 后代组件（任意层级） -->
2  <script setup lang="ts">
3    import { inject } from 'vue'
4
5    const theme = inject('theme')
6    const user = inject('user')
7    const setTheme = inject('setTheme')
8
9    function toggleTheme() {
10     setTheme?.(theme.value === 'light' ? 'dark' : 'light')
11   }
12 </script>
13
14 <template>
15   <p>Theme: {{ theme }}</p>
16   <p>User: {{ user?.name }}</p>
17   <button @click="toggleTheme">切换主题</button>
18 </template>
```

vue

TIP

Provide/Inject 适合深层嵌套的组件通信，避免 Props 逐层传递（Prop Drilling）。

5.3 Mitt 事件总线（兄弟组件通信）

```
1  npm install mitt
```

⌨️ Bash

```
1  // eventBus.ts
2  import mitt from 'mitt'
3
4  export const emitter = mitt()
5
6  // 类型定义
7  type Events = {
8    'user-login': { userId: number; username: string }
9    'cart-update': { itemCount: number }
10   'theme-change': string
11 }
12
13 export type Emitter = typeof emitter
```

📘 TypeScript

```
1 <!-- 组件 A: 发送事件 --> vue
2 <script setup lang="ts">
3 import { emitter } from '@eventBus'
4
5 function login() {
6   emitter.emit('user-login', {
7     userId: 1,
8     username: 'Alice'
9   })
10 }
11 </script>
```

```
1 <!-- 组件 B: 接收事件 --> vue
2 <script setup lang="ts">
3 import { onMounted, onUnmounted } from 'vue'
4 import { emitter } from '@eventBus'
5
6 onMounted(() => {
7   emitter.on('user-login', ({ userId, username }) => {
8     console.log(`User ${username} (${userId}) logged in`)
9   })
10 })
11
12 onUnmounted(() => {
13   emitter.off('user-login') // 清理事件监听
14 })
15 </script>
```

CAUTION

事件总线适合小型应用，大型应用推荐使用 Pinia 进行状态管理。

5.4 Pinia（全局状态管理）

```
1 // stores/user.ts TypeScript
2 import { defineStore } from 'pinia'
3 import { ref } from 'vue'
4
5 export const useUserStore = defineStore('user', () => {
6   const user = ref<{ name: string; email: string } | null>(null)
7   const isLoggedIn = ref(false)
8
9   function login(name: string, email: string) {
10     user.value = { name, email }
11     isLoggedIn.value = true
12   }
13
14   function logout() {
15     user.value = null
16     isLoggedIn.value = false
17   }
18
19   return { user, isLoggedIn, login, logout }
```

```
20  })
```

```
1  <!-- 任意组件中使用 -->
2  <script setup lang="ts">
3  import { useUserStore } from '@/stores/user'
4
5  const userStore = useUserStore()
6
7  function handleLogin() {
8    userStore.login('Alice', 'alice@example.com')
9  }
10 </script>
11
12 <template>
13   <div v-if="userStore.isLoggedIn">
14     <p>Welcome, {{ userStore.user?.name }}</p>
15     <button @click="userStore.logout">Logout</button>
16   </div>
17   <button v-else @click="handleLogin">Login</button>
18 </template>
```

NOTE

Pinia 是 Vue 3 推荐的状态管理方案，替代了 Vuex。

本章完

Chapter 4

指令与过渡动画

Vue 提供了强大的指令系统和过渡动画系统，让 DOM 操作和动画效果变得简单易用。

1 内置指令

1.1 v-if / v-show

```
1 <script setup lang="ts">
2   import { ref } from 'vue'
3
4   const isVisible = ref(true)
5   const userType = ref('admin')
6 </script>
7
8 <template>
9   <!-- v-if: 条件渲染，元素会被销毁/重建 -->
10  <div v-if="isVisible">显示的内容</div>
11
12  <!-- v-else-if / v-else -->
13  <div v-if="userType === 'admin'">管理员界面</div>
14  <div v-else-if="userType === 'user'">用户界面</div>
15  <div v-else>访客界面</div>
16
17  <!-- v-show: 条件显示，元素始终存在，只切换 display -->
18  <div v-show="isVisible">显示的内容</div>
19 </template>
```

对比：

特性	v-if	v-show
渲染方式	条件渲染	条件显示
DOM 元素	销毁/重建	始终存在
CSS display	不涉及	none/block
初始开销	低（如果不显示）	高（始终渲染）
切换开销	高	低
适用场景	不频繁切换	频繁切换

TIP

频繁切换用 v-show，很少切换用 v-if。

1.2 v-for

```
1 <script setup lang="ts">
2   import { ref } from 'vue'
3
4   interface User {
5     id: number
6     name: string
7     email: string
8   }
9
10  const users = ref<User[]>([
11    { id: 1, name: 'Alice', email: 'alice@example.com' },
12    { id: 2, name: 'Bob', email: 'bob@example.com' },
13  ])
14
15  const numbers = ref([1, 2, 3, 4, 5])
16  const object = ref({ name: 'Alice', age: 25 })
17 </script>
18
19 <template>
20   <!-- 数组遍历 -->
21   <ul>
22     <li v-for="(user, index) in users" :key="user.id">
23       {{ index + 1 }}. {{ user.name }} - {{ user.email }}
24     </li>
25   </ul>
26
27   <!-- 数字范围 -->
28   <span v-for="n in 10" :key="n">{{ n }}</span>
29
30   <!-- 对象遍历 -->
31   <div v-for="(value, key) in object" :key="key">
32     {{ key }}: {{ value }}
33   </div>
34
35   <!-- 字符串遍历 -->
36   <span v-for="char in 'Hello'" :key="char">{{ char }}</span>
37 </template>
```

CAUTION

v-for 必须绑定 :key, 且 key 应该是唯一且稳定的标识符 (如 ID), 不要用 index。

1.3 v-bind / v-on

```
1 <script setup lang="ts">
2   import { ref } from 'vue'
3
4   const imageUrl = ref('/path/to/image.jpg')
5   const isDisabled = ref(false)
6   const customClass = ref('active')
```

```

7
8  function handleClick(event: MouseEvent) {
9    console.log('Clicked!', event)
10 }
11
12 function handleKeydown(event: KeyboardEvent) {
13   if (event.key === 'Enter') {
14     console.log('Enter pressed')
15   }
16 }
17 </script>
18
19 <template>
20   <!-- v-bind: 动态绑定属性 -->
21   
22   <button :disabled="isDisabled">按钮</button>
23   <div :class="customClass">内容</div>
24
25   <!-- 对象语法绑定 class -->
26   <div :class="{ active: isActive, 'text-danger': hasError }">内容</div>
27
28   <!-- 数组语法绑定 class -->
29   <div :class="[baseClass, errorClass]">内容</div>
30
31   <!-- 对象语法绑定 style -->
32   <div :style="{ color: activeColor, fontSize: fontSize + 'px' }">内容</div>
33
34   <!-- v-on: 事件监听 -->
35   <button @click="handleClick">点击</button>
36   <input @keydown="handleKeydown" />
37
38   <!-- 事件修饰符 -->
39   <form @submit.prevent="handleSubmit">表单</form>
40   <button @click.stop="handleClick">阻止冒泡</button>
41   <a @click.once="handleClick">只触发一次</a>
42
43   <!-- 按键修饰符 -->
44   <input @keyup.enter="handleSubmit" />
45   <input @keyup.ctrl.enter="handleSubmit" />
46
47   <!-- 简写语法 -->
48   
49 </template>

```

NOTE

`:` 是 `v-bind` 的简写, `@` 是 `v-on` 的简写。

1.4 v-model

```

1 <script setup lang="ts">
2 import { ref } from 'vue'

```

vue

```
3
4  const text = ref('')
5  const checked = ref(false)
6  const selected = ref('option1')
7  const number = ref(0)
8  </script>
9
10 <template>
11   <!-- 文本输入 -->
12   <input v-model="text" type="text" />
13
14   <!-- 复选框 -->
15   <input v-model="checked" type="checkbox" />
16
17   <!-- 单选框 -->
18   <input v-model="selected" type="radio" value="option1" />
19   <input v-model="selected" type="radio" value="option2" />
20
21   <!-- 下拉选择 -->
22   <select v-model="selected">
23     <option value="option1">选项 1</option>
24     <option value="option2">选项 2</option>
25   </select>
26
27   <!-- 修饰符 -->
28   <input v-model.trim="text" /> <!-- 自动去除首尾空格 -->
29   <input v-model.number="number" type="number" /> <!-- 自动转换为数字 -->
30   <input v-model.lazy="text" /> <!-- change 事件而非 input 事件 -->
31 </template>
```

自定义组件的 v-model:

```
1  <!-- CustomInput.vue -->
2  <script setup lang="ts">
3    const props = defineProps<{
4      modelValue: string
5    }>()
6
7    const emit = defineEmits<{
8      (e: 'update:modelValue', value: string): void
9    }>()
10
11    function updateValue(event: Event) {
12      const target = event.target as HTMLInputElement
13      emit('update:modelValue', target.value)
14    }
15  </script>
16
17  <template>
18    <input :value="modelValue" @input="updateValue" />
19  </template>
```

```
1  <!-- 使用 -->
```

vue

vue

```
2 <script setup lang="ts">
3 import { ref } from 'vue'
4 import CustomInput from './CustomInput.vue'
5
6 const inputValue = ref('')
7 </script>
8
9 <template>
10   <CustomInput v-model="inputValue" />
11 </template>
```

1.5 v-slot

```
1 <!-- Card.vue -->
2 <template>
3   <div class="card">
4     <header>
5       <slot name="header">默认标题</slot>
6     </header>
7     <main>
8       <slot>默认内容</slot>
9     </main>
10    <footer>
11      <slot name="footer">默认底部</slot>
12    </footer>
13  </div>
14 </template>
```

使用：

```
1 <Card>
2   <template #header>
3     <h2>标题</h2>
4   </template>
5
6   <p>内容</p>
7
8   <template #footer>
9     <button>按钮</button>
10  </template>
11 </Card>
```

TIP

是 v-slot: 的简写，#default 可以省略。

2 自定义指令

2.1 指令注册

```
1 // directives/focus.ts
2 import type { Directive } from 'vue'
3
4 export const focus: Directive = {
5   mounted: (el) => el.focus()
6 }
```

```
1 // main.ts
2 import { createApp } from 'vue'
3 import App from './App.vue'
4 import { focus } from './directives/focus'
5
6 const app = createApp(App)
7 app.directive('focus', focus)
8 app.mount('#app')
```

局部注册:

```
1 <script setup lang="ts">
2   import type { Directive } from 'vue'
3
4   const focus: Directive = {
5     mounted: (el) => el.focus()
6   }
7 </script>
8
9 <template>
10   <input v-focus />
11 </template>
```

2.2 钩子函数

```
1 import type { DirectiveBinding } from 'vue'
2
3 const myDirective = {
4   // 在绑定元素的 attribute 前或事件监听器应用前调用
5   created(el: HTMLElement, binding: DirectiveBinding, vnode, prevVnode) {
6     console.log('created')
7   },
8
9   // 在元素被插入到 DOM 前调用
10  beforeMount(el, binding, vnode, prevVnode) {
11    console.log('beforeMount')
12  },
13
14  // 在绑定元素的父组件及他自己的所有子节点都挂载完成后调用
15  mounted(el, binding, vnode, prevVnode) {
16    console.log('mounted')
17  },
18
19  // 绑定元素的父组件更新前调用
20  beforeUpdate(el, binding, vnode, prevVnode) {
```

```
21   console.log('beforeUpdate')
22 },
23
24 // 在绑定元素的父组件及他自己的所有子节点都更新后调用
25 updated(el, binding, vnode, prevVnode) {
26   console.log('updated')
27 },
28
29 // 绑定元素的父组件卸载前调用
30 beforeUnmount(el, binding, vnode, prevVnode) {
31   console.log('beforeUnmount')
32 },
33
34 // 绑定元素的父组件卸载后调用
35 unmounted(el, binding, vnode, prevVnode) {
36   console.log('unmounted')
37 },
38 }
```

NOTE

Vue 3 中自定义指令的钩子函数名称与组件生命周期钩子一致。

2.3 实战示例

自动聚焦：

```
1 // directives/focus.ts
2 import type { Directive } from 'vue'
3
4 export const focus: Directive<HTMLInputElement> = {
5   mounted: (el) => {
6     el.focus()
7   }
8 }
```

 TypeScript

```
1 <template>
2   <input v-focus placeholder="自动聚焦" />
3 </template>
```

 vue

权限控制：

```
1 // directives/permission.ts
2 import type { Directive } from 'vue'
3 import { useUserStore } from '@/stores/user'
4
5 export const permission: Directive = {
6   mounted(el: HTMLElement, binding) {
7     const userStore = useUserStore()
8     const requiredRole = binding.value
9
10    if (!userStore.hasPermission(requiredRole)) {
11      el.parentNode?.removeChild(el)
12    }
13  }
14 }
```

 TypeScript

```
13   }  
14 }
```

```
1 <template>  
2   <button v-permission="'admin'">仅管理员可见</button>  
3   <button v-permission="['admin', 'editor']">管理员或编辑可见</button>  
4 </template>
```

vue

防抖:

```
1 // directives/debounce.ts  
2 import type { Directive } from 'vue'  
3  
4 export const debounce: Directive = {  
5   mounted(el: HTMLElement, binding) {  
6     let timeoutId: number  
7  
8     const handler = (event: Event) => {  
9       clearTimeout(timeoutId)  
10      timeoutId = setTimeout(() => {  
11        binding.value(event)  
12      }, 300) as unknown as number  
13    }  
14  
15    el.addEventListener('input', handler)  
16  
17    // 存储清理函数  
18    ;(el as any)._debounceCleanup = () => {  
19      el.removeEventListener('input', handler)  
20      clearTimeout(timeoutId)  
21    }  
22  },  
23  
24  unmounted(el: HTMLElement) {  
25    ;(el as any)._debounceCleanup?.()  
26  }  
27 }
```

TypeScript

```
1 <script setup lang="ts">  
2 import { debounce } from '@directives/debounce'  
3  
4 function handleSearch(event: Event) {  
5   const target = event.target as HTMLInputElement  
6   console.log('Search:', target.value)  
7 }  
8 </script>  
9  
10 <template>  
11   <input v-debounce="handleSearch" placeholder="搜索..." />  
12 </template>
```

vue

TIP

自定义指令适合封装 DOM 操作相关的逻辑，如聚焦、滚动、拖拽等。

3 过渡系统

3.1 <Transition> 基础

```
1 <script setup lang="ts">
2   import { ref } from 'vue'
3
4   const show = ref(false)
5 </script>
6
7 <template>
8   <button @click="show = !show">切换</button>
9
10  <Transition name="fade">
11    <div v-if="show" class="box">内容</div>
12  </Transition>
13 </template>
14
15 <style>
16  /* fade 过渡 */
17  .fade-enter-active,
18  .fade-leave-active {
19    transition: opacity 0.3s ease;
20  }
21
22  .fade-enter-from,
23  .fade-leave-to {
24    opacity: 0;
25  }
26 </style>
```

过渡类名：

类名	时机	说明
v-enter-from	进入前	起始状态
v-enter-active	进入中	激活状态，定义过渡
v-enter-to	进入后	结束状态
v-leave-from	离开前	起始状态
v-leave-active	离开中	激活状态，定义过渡
v-leave-to	离开后	结束状态

NOTE

如果给 `<Transition>` 设置了 `name` 属性（如 `name="fade"`），则类名前缀从 `v-` 变为 `fade-`。

3.2 CSS 过渡/动画

淡入淡出：


```
1 .fade-enter-active,
2 .fade-leave-active {
3   transition: opacity 0.3s ease;
4 }
5
6 .fade-enter-from,
7 .fade-leave-to {
8   opacity: 0;
9 }
```

滑动:

```
1 .slide-enter-active,
2 .slide-leave-active {
3   transition: transform 0.3s ease;
4 }
5
6 .slide-enter-from {
7   transform: translateX(-100%);
8 }
9
10 .slide-leave-to {
11   transform: translateX(100%);
12 }
```

缩放:

```
1 .scale-enter-active,
2 .scale-leave-active {
3   transition: transform 0.3s ease, opacity 0.3s ease;
4 }
5
6 .scale-enter-from,
7 .scale-leave-to {
8   transform: scale(0);
9   opacity: 0;
10 }
```

使用 CSS Animation:

```
1 <Transition name="bounce">
2   <div v-if="show">内容</div>
3 </Transition>
4
5 <style>
6   .bounce-enter-active {
7     animation: bounce-in 0.5s;
8   }
9
10  .bounce-leave-active {
11    animation: bounce-in 0.5s reverse;
12  }
13
14  @keyframes bounce-in {
```

```
15 0% {
16   transform: scale(0);
17 }
18 50% {
19   transform: scale(1.25);
20 }
21 100% {
22   transform: scale(1);
23 }
24 }
25 </style>
```

TIP

Transition 支持 CSS transition 和 animation, Vue 会自动检测并使用。

3.3 JavaScript 钩子

```
1 <script setup lang="ts">
2 import { ref } from 'vue'
3 import gsap from 'gsap'
4
5 const show = ref(false)
6
7 function onBeforeEnter(el: Element) {
8   gsap.set(el, {
9     scaleX: 0.25,
10    scaleY: 0.25,
11    opacity: 1
12  })
13 }
14
15 function onEnter(el: Element, done: () => void) {
16   gsap.to(el, {
17     duration: 1,
18     scaleX: 1,
19     scaleY: 1,
20     ease: "elastic.inOut(2.5, 1)",
21     onComplete: done
22   })
23 }
24
25 function onLeave(el: Element, done: () => void) {
26   gsap.to(el, {
27     duration: 0.7,
28     scaleX: 1,
29     scaleY: 1,
30     x: 300,
31     ease: "elastic.inOut(2.5, 1)"
32   })
33   gsap.to(el, {
34     duration: 0.2,
```

vue

```

35     delay: 0.5,
36     opacity: 0,
37     onComplete: done
38   })
39 }
40 </script>
41
42 <template>
43   <button @click="show = !show">切换</button>
44
45   <Transition
46     @before-enter="onBeforeEnter"
47     @enter="onEnter"
48     @leave="onLeave"
49     :css="false"
50   >
51     <div v-if="show" class="box">GSAP 动画</div>
52   </Transition>
53 </template>

```

JavaScript 钩子列表：

钩子	参数	说明
@before-enter	el	进入前
@enter	el, done	进入中，done 是回调
@after-enter	el	进入后
@enter-cancelled	el	进入取消
@before-leave	el	离开前
@leave	el, done	离开中，done 是回调
@after-leave	el	离开后
@leave-cancelled	el	离开取消

CAUTION

使用 JavaScript 钩子时，需要设置 `:css="false"`，否则 Vue 会同时应用 CSS 过渡。

4 列表过渡

4.1

基础

```

1 <script setup lang="ts">
2 import { ref } from 'vue'
3
4 let nextId = 1
5 const items = ref([

```

vue

```
6   { id: nextId++, text: 'Item 1' },
7   { id: nextId++, text: 'Item 2' },
8   { id: nextId++, text: 'Item 3' },
9 ])
10
11 function addItem() {
12   items.value.push({ id: nextId++, text: `Item ${nextId - 1}` })
13 }
14
15 function removeItem(item: any) {
16   const index = items.value.indexOf(item)
17   if (index > -1) {
18     items.value.splice(index, 1)
19   }
20 }
21 </script>
22
23 <template>
24   <button @click="addItem">添加</button>
25
26   <TransitionGroup name="list" tag="ul">
27     <li v-for="item in items" :key="item.id" @click="removeItem(item)">
28       {{ item.text }}
29     </li>
30   </TransitionGroup>
31 </template>
32
33 <style>
34   .list-enter-active,
35   .list-leave-active {
36     transition: all 0.3s ease;
37   }
38
39   .list-enter-from,
40   .list-leave-to {
41     opacity: 0;
42     transform: translateX(30px);
43   }
44
45   .list-leave-active {
46     position: absolute; /* 防止其他元素跳动 */
47   }
48 </style>
```

NOTE

TransitionGroup 与 Transition 的区别：

- TransitionGroup 用于列表，Transition 用于单个元素
- TransitionGroup 没有 mode 属性
- TransitionGroup 需要 tag 或 appear 属性

4.2 FLIP 动画

FLIP (First, Last, Invert, Play) 是一种高性能的动画技术。

```
1  <script setup lang="ts">
2    import { ref } from 'vue'
3
4    const items = ref([1, 2, 3, 4, 5])
5
6    function shuffle() {
7      items.value = items.value.sort(() => Math.random() - 0.5)
8    }
9  </script>
10
11 <template>
12   <button @click="shuffle">打乱顺序</button>
13
14   <TransitionGroup name="flip" tag="div" class="container">
15     <div v-for="item in items" :key="item" class="item">
16       {{ item }}
17     </div>
18   </TransitionGroup>
19 </template>
20
21 <style>
22   .flip-move {
23     transition: transform 0.3s ease;
24   }
25
26   .container {
27     display: flex;
28     gap: 10px;
29   }
30
31   .item {
32     width: 50px;
33     height: 50px;
34     background: #42b983;
35     color: white;
36     display: flex;
37     align-items: center;
38     justify-content: center;
39     border-radius: 4px;
40   }
41 </style>
```

FLIP 原理：

1. First: 记录元素的初始位置
2. Last: 记录元素的最终位置
3. Invert: 计算位置差值，应用 transform 将元素移回初始位置
4. Play: 移除 transform，让元素平滑移动到最终位置

TIP

Vue 的 TransitionGroup 自动实现了 FLIP 动画，只需添加 `.move` 类即可。

4.3 性能优化

使用 will-change:

```
1 .list-item {  
2   will-change: transform, opacity;  
3 }
```

避免布局抖动:

```
1 .list-leave-active {  
2   position: absolute; /* 防止其他元素重新布局 */  
3 }
```

使用 transform 而非 top/left:

```
1 /* ✓ 好: 使用 transform (GPU 加速) */  
2 .item {  
3   transform: translateX(0);  
4   transition: transform 0.3s;  
5 }  
6  
7 /* ✗ 差: 使用 top/left (触发重排) */  
8 .item {  
9   top: 0;  
10  transition: top 0.3s;  
11 }
```

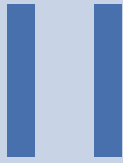
限制动画数量:

```
1 <!-- 只动画可见区域内的元素 -->  
2 <TransitionGroup name="list">  
3   <li v-for="item in visibleItems" :key="item.id">  
4     {{ item.text }}  
5   </li>  
6 </TransitionGroup>
```

CAUTION

过多的过渡动画会影响性能，特别是在移动设备上。谨慎使用，并进行性能测试。

本章完



Vue 官方生态

5.	Vue Router 路由管理 ·····	40
6.	Pinia 状态管理 ·····	41
7.	数据持久化 ·····	42
8.	HTTP 请求与 API 管理 ·····	43

Chapter 5

Vue Router 路由管理

Chapter 6

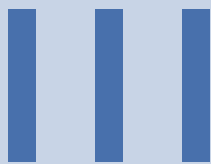
Pinia 状态管理

Chapter 7

数据持久化

Chapter 8

HTTP 请求与 API 管理



现代 UI 库与 Shadcn-vue

9. UI 库演进与 Shadcn 崛起	45
10. Shadcn-vue 快速上手	54
11. Shadcn-vue 核心组件	55
12. Shadcn-vue 高级用法	56
13. 其他现代 UI 方案	57

Chapter 9

UI 库演进与 Shadcn 崛起

UI 组件库是前端开发的核心基础设施。从传统的 Element UI、Ant Design 到现代的 Shadcn-vue，UI 库的理念和实现方式发生了巨大变化。

1 传统 UI 库困境

在 Shadcn 出现之前，主流 UI 库（Element UI、Ant Design、Vuetify 等）面临诸多问题。

1.1 体积臃肿

```
1 // Element Plus 完整引入
2 import ElementPlus from 'element-plus'
3 import 'element-plus/dist/index.css'
4
5 // Bundle 大小分析
6 // element-plus: ~1.2MB (uncompressed)
7 // 即使 Tree-Shaking 后，仍然有数百 KB
```

问题分析：

问题	影响	示例
完整 CSS	首屏加载慢	~400KB CSS
未使用的组件	Bundle 过大	只用了 Button，却打包了所有组件
依赖过多	间接依赖膨胀	dayjs、lodash 等

CAUTION

传统 UI 库即使用了按需导入，仍然会引入大量未使用的样式和工具函数。

1.2 样式覆盖困难

```
1 /* 覆盖 Element UI 样式 */
2 .el-button {
3   /* 需要 !important 或更高优先级 */
4   background-color: #ff0000 !important;
5 }
6
7 /* 深度选择器 */
8 ::v-deep .el-input__inner {
```

```
9   border-radius: 8px;
10 }
11
12 /* Scoped CSS 中的覆盖 */
13 <style scoped>
14   .my-component :deep(.el-button) {
15     color: blue;
16   }
17 </style>
```

痛点:

1. CSS 优先级战争: 需要 `!important` 或复杂的选择器
2. Scoped CSS 限制: 需要使用 `::v-deep`、`:deep()` 等穿透语法
3. 主题定制复杂: 需要修改 Sass 变量或提供完整的主题包
4. 版本升级风险: 内部类名可能改变, 导致样式失效

NOTE

传统 UI 库的样式封装过度, 导致定制化成本极高。

1.3 定制性差

```
1 <!-- Element UI Button -->
2 <el-button type="primary">按钮</el-button>
3
4 <!-- 想要完全自定义? 很难 -->
5 <!-- 只能覆盖样式, 无法改变结构 -->
6 <!-- 无法添加额外的 DOM 节点 -->
7 <!-- 无法控制内部的 class 命名 -->
```

vue

限制:

- 固定 DOM 结构: 无法修改组件内部的 HTML 结构
- 固定的 Class 命名: BEM 命名规范, 无法自定义
- 有限的 Props: 只能通过提供的 props 配置, 无法扩展
- 黑盒实现: 内部逻辑不透明, 难以调试

TIP

传统 UI 库适合快速原型开发, 但不适合高度定制化的产品。

2 Shadcn 理念

Shadcn 不是传统意义上的 UI 库, 而是一种全新的组件构建理念。

2.1 不是组件库而是代码集合

```
1 # 传统 UI 库: npm install
2 npm install element-plus
3
```

Bash

```

4 # Shadcn: 复制代码到你的项目
5 npx shadcn-vue@latest add button
6
7 # 结果: 组件代码直接复制到 src/components/ui/button.vue
8 # 你拥有完全的代码控制权

```

核心理念对比:

特性	传统 UI 库	Shadcn
安装方式	npm install	复制代码到项目
代码所有权	第三方依赖	你的代码
定制难度	高 (覆盖样式)	低 (直接修改代码)
更新方式	npm update	手动合并或重新添加
Bundle 大小	包含所有组件	只包含使用的组件
学习曲线	学习 API	学习代码本身

NOTE

Shadcn 的本质是「可复制的代码片段」, 而不是「安装的依赖包」。

2.2 Copy-Paste workflow

```

1 # 1. 初始化 (一次性)
2 npx shadcn-vue@latest init
3
4 # 2. 添加组件 (按需)
5 npx shadcn-vue@latest add button
6 npx shadcn-vue@latest add dialog
7 npx shadcn-vue@latest add table
8
9 # 3. 代码出现在项目中
10 src/
11 |— components/
12 |   |— ui/
13 |       |— button/
14 |           |— Button.vue
15 |           |— index.ts
16 |       |— dialog/
17 |           |— Dialog.vue
18 |           |— ...
19 |       |— table/
20 |           |— ...

```

 Bash

优势:

1. 完全可控: 代码在你的项目中, 可以随意修改
2. 零抽象: 没有黑盒, 所有逻辑可见
3. 类型安全: TypeScript 支持, IDE 自动补全
4. 无运行时依赖: 不需要额外的 JS bundle

2.3 基于 Tailwind CSS

```
1 <!-- Shadcn Button 源码 -->
2 <template>
3   <button
4     :class="cn(
5       'inline-flex items-center justify-center rounded-md text-sm font-medium',
6       'ring-offset-background transition-colors',
7       'focus-visible:outline-none focus-visible:ring-2',
8       'focus-visible:ring-ring focus-visible:ring-offset-2',
9       'disabled:pointer-events-none disabled:opacity-50',
10      variantClass,
11      sizeClass,
12      $attrs.class
13    )"
14  >
15    <slot />
16  </button>
17 </template>
18
19 <script setup lang="ts">
20   import { cn } from '@lib/utils'
21   // ... props 定义
22 </script>
```

Tailwind 优势：

- Utility-first：直接使用原子类，无需编写 CSS
- 设计系统：通过 `tailwind.config.js` 统一定义颜色、间距等
- 响应式：内置响应式前缀（`sm:`、`md:`、`lg:`）
- 暗色模式：`dark:` 前缀轻松实现主题切换
- Tree-Shaking：PurgeCSS 自动移除未使用的类

TIP

Tailwind CSS + Shadcn = 极致的样式定制能力 + 零运行时开销

3 Shadcn 优势

3.1 零运行时开销

```
1 // 传统 UI 库
2 import { ElButton } from 'element-plus'
3 // → 引入整个 Element Plus 运行时
4 // → 包括事件处理、状态管理、动画系统等
5
6 // Shadcn
7 import Button from '@components/ui/button/Button.vue'
8 // → 只引入一个 Vue 组件
9 // → 没有任何额外的运行时依赖
```


Bundle 对比:

方案	Button 组件	额外依赖	总大小
Element Plus	~5KB	~1.2MB	~1.2MB
Ant Design Vue	~4KB	~800KB	~800KB
Shadcn-vue	~2KB	0KB	~2KB

CAUTION

Shadcn 的 Bundle 大小只包含你实际使用的组件，没有隐藏成本。

3.2 无障碍访问 (a11y)

Shadcn 基于 Radix UI / Reka UI，这些 Headless UI 库专注于无障碍。

```

1  <!-- Reka UI Dialog (Shadcn 底层) -->
2  <DialogRoot>
3    <DialogTrigger as-child>
4      <Button>打开对话框</Button>
5    </DialogTrigger>
6
7    <DialogPortal>
8      <DialogOverlay class="fixed inset-0 bg-black/50" />
9      <DialogContent
10        class="fixed top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2"
11        role="dialog"
12        aria-modal="true"
13        aria-labelledby="dialog-title"
14      >
15        <DialogTitle id="dialog-title">标题</DialogTitle>
16        <DialogDescription>描述</DialogDescription>
17
18        <!-- 自动焦点管理 -->
19        <!-- 自动 ESC 关闭 -->
20        <!-- 自动 Trap Focus -->
21      </DialogContent>
22    </DialogPortal>
23  </DialogRoot>
  
```

a11y 特性:

- ✓ 键盘导航: Tab、Enter、Escape 等快捷键
- ✓ 焦点管理: 自动聚焦、焦点陷阱
- ✓ ARIA 属性: role、aria-label、aria-describedby
- ✓ 屏幕阅读器: 完整的语义化支持
- ✓ 颜色对比度: 符合 WCAG 标准

NOTE

无障碍访问不是可选功能，而是现代 Web 应用的必备要求。

3.3 主题定制

```
1 // tailwind.config.js
2 module.exports = {
3   theme: {
4     extend: {
5       colors: {
6         border: 'hsl(var(--border))',
7         input: 'hsl(var(--input))',
8         ring: 'hsl(var(--ring))',
9         background: 'hsl(var(--background))',
10        foreground: 'hsl(var(--foreground))',
11        primary: {
12          DEFAULT: 'hsl(var(--primary))',
13          foreground: 'hsl(var(--primary-foreground))',
14        },
15        // ... 更多颜色
16      },
17    },
18  },
19 }
```

```
1 /* globals.css */
2 :root {
3   --background: 0 0% 100%;
4   --foreground: 222.2 84% 4.9%;
5   --primary: 222.2 47.4% 11.2%;
6   --primary-foreground: 210 40% 98%;
7   /* ... */
8 }
9
10 .dark {
11   --background: 222.2 84% 4.9%;
12   --foreground: 210 40% 98%;
13   --primary: 210 40% 98%;
14   --primary-foreground: 222.2 47.4% 11.2%;
15   /* ... */
16 }
```

定制流程：

1. 修改 CSS 变量（全局主题）
2. 修改 `tailwind.config.js`（设计系统）
3. 直接修改组件代码（局部定制）

TIP

三层定制能力：全局主题 → 设计系统 → 组件代码，灵活性极高。

3.4 Tree-Shaking 友好

```
1 // Vite + Rollup 自动 Tree-Shaking
2
```

```
3 // 只导入使用的组件
4 import Button from '@/components/ui/button/Button.vue'
5 import Dialog from '@/components/ui/dialog/Dialog.vue'
6
7 // 未导入的组件不会被打包
8 // Table、Calendar、DatePicker 等不会出现在最终 bundle 中
```

优化效果：

- 按需加载：只打包实际使用的组件
- 代码分割：配合动态导入实现路由级分割
- CSS 清理：PurgeCSS 移除未使用的 Tailwind 类
- 无死代码：没有未使用的导出或副作用

4 生态对比

4.1 Radix UI (React Headless UI)

```
1 // Radix UI (React)
2 import * as Dialog from '@radix-ui/react-dialog'
3
4 function MyDialog() {
5   return (
6     <Dialog.Root>
7       <Dialog.Trigger>Open</Dialog.Trigger>
8       <Dialog.Portal>
9         <Dialog.Overlay />
10        <Dialog.Content>
11          <Dialog.Title>Title</Dialog.Title>
12          <Dialog.Description>Description</Dialog.Description>
13        </Dialog.Content>
14      </Dialog.Portal>
15    </Dialog.Root>
16  )
17 }
```

特点：

- Headless：无样式，只提供行为和 a11y
- React 专用：不支持 Vue
- 组合式 API：Compound Components 模式
- Shadcn/ui 的基础：Shadcn React 版基于 Radix

4.2 Reka UI (Vue Headless UI)

```
1 <!-- Reka UI (Vue) -->
2 <script setup>
3   import {
4     DialogRoot,
5     DialogTrigger,
6     DialogPortal,
```

```
7   DialogOverlay,
8   DialogContent,
9   DialogTitle,
10  DialogDescription,
11 } from 'reka-ui'
12 </script>
13
14 <template>
15   <DialogRoot>
16     <DialogTrigger as-child>
17       <Button>打开</Button>
18     </DialogTrigger>
19
20     <DialogPortal>
21       <DialogOverlay />
22       <DialogContent>
23         <DialogTitle>标题</DialogTitle>
24         <DialogDescription>描述</DialogDescription>
25       </DialogContent>
26     </DialogPortal>
27   </DialogRoot>
28 </template>
```

特点：

- Vue 版 Radix：相同的 API 设计
- Headless：无样式，专注行为
- Composition API：完美适配 Vue 3
- Shadcn-vue 的基础：Shadcn Vue 版基于 Reka

NOTE

Reka UI 是 Radix UI 的 Vue 移植，由同一团队维护。

4.3 Shadcn/ui (React) vs Shadcn-vue (Vue)

特性	Shadcn/ui (React)	Shadcn-vue (Vue)
基础库	Radix UI	Reka UI
语言	TypeScript + JSX	TypeScript + SFC
样式	Tailwind CSS	Tailwind CSS
CLI 工具	shadcn-ui	shadcn-vue
社区规模	大 (React 生态)	增长中 (Vue 生态)
组件数量	50+	40+
成熟度	高	中高

共同点：

-  相同的理念：Copy-Paste、完全可控
-  相同的技术栈：Tailwind CSS + Headless UI
-  相同的 CLI：`npx shadcn add <component>`

-  相同的定制能力：直接修改源码

差异：

- React 版更早推出，社区更成熟
- Vue 版正在快速追赶，组件覆盖率已达 80%+
- API 设计风格一致，降低跨框架学习成本

本章完

Chapter 10

Shadcn-vue 快速上手

Chapter 11

Shadcn-vue 核心组件

Chapter 12

Shadcn-vue 高级用法

Chapter 13

其他现代 UI 方案

IV

工程化与最佳实践

14. TypeScript 集成 ·····	59
15. Vite 构建优化 ·····	60
16. 测试与质量保障 ·····	61
17. 性能优化 ·····	62

Chapter 14

TypeScript 集成

Chapter 15

Vite 构建优化

Chapter 16

测试与质量保障

Chapter 17

性能优化



Vue 2 兼容与迁移

18. Options API 回顾 ·····	64
19. Vue 2 → Vue 3 迁移 ·····	65

Chapter 18

Options API 回顾

Chapter 19

Vue 2 → Vue 3 迁移
